STUDENT ATTENDANCE MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

ANCHAL KUMARI
UID: 25BCD10251
Class/Section: 25BBCD4A

in partial fulfillment for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS (BCA)

IN
COMPUTER APPLICATIONS

Chandigarh University

October 2026

BONAFIDE CERTIFICATE

Certified that this project report entitled **"STUDENT ATTENDANCE MANAGEMENT SYSTEM"** is the bonafide work of **Anchal Kumari (UID: 25BCD10251, Class/Section: 25BBCD4A)**, who carried out the project work under the guidance and supervision of the undersigned.

_____	_____
HEAD OF THE DEPARTMENT	**PROJECT SUPERVISOR**
Name: _____	Name: _____

Submitted for the project viva-voce examination held on: _____

_____	_____
INTERNAL EXAMINER	**EXTERNAL EXAMINER**

DECLARATION

I, **Anchal Kumari**, hereby declare that the project report entitled **“Student Attendance Management System”** has been prepared by me as an academic project. The report describes the proposed design and a console-based implementation approach using C++. The work has been prepared for educational purposes, and all external sources referred to are listed in the References section.

Place: Chandigarh University
Date: October 2026

Signature of Student: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to Chandigarh University for providing the academic environment and resources required to undertake this project. I am thankful to the Head of the Department and my project supervisor for their guidance, suggestions and encouragement throughout the preparation of this report.

I also thank my classmates, friends and family members for their support. Finally, I acknowledge the authors and documentation resources that helped me understand C++ programming, data handling and the basic principles of attendance management systems.

TABLE OF CONTENTS

Abstract 5

Graphical Abstract 6

Abbreviations and Symbols 7

CHAPTER 1. INTRODUCTION 8

1.1 Need and contemporary issue 8

1.2 Identification of problem 8

1.3 Identification of tasks 9

1.4 Project timeline 9

1.5 Organization of report 9

CHAPTER 2. LITERATURE REVIEW / BACKGROUND STUDY 10

2.1 Development of attendance recording 10

2.2 Existing solutions 10

2.3 Comparative analysis 11

2.4 Review summary 11

2.5 Problem definition 11

2.6 Goals and objectives 11

CHAPTER 3. DESIGN FLOW / PROCESS 12

3.1 Features and specifications 12

3.2 Design constraints 12

3.3 Feature finalization 13

3.4 Alternative designs 13

3.5 Design selection 13

3.6 Methodology 14

CHAPTER 4. RESULTS ANALYSIS AND VALIDATION 15

4.1 Implementation approach 15

4.2 Test cases and validation 16

CHAPTER 5. CONCLUSION AND FUTURE WORK 17

References 18

Appendix and User Manual 19

LIST OF FIGURES

Figure 3.1: Overall system architecture

Figure 3.2: Attendance recording flowchart

Figure 4.1: Sample console menu and output

LIST OF TABLES

Table 1.1: Project timeline

Table 2.1: Comparison of existing attendance methods

Table 3.1: Functional requirements

Table 4.1: Test cases and expected results

ABSTRACT

Attendance is an important part of academic administration. In many classrooms, attendance is recorded manually in registers or separate spreadsheets. Such methods can require time, make searching older records difficult and increase the possibility of calculation or entry errors. This project presents a Student Attendance Management System intended to make the recording and review of student attendance more organized.

The proposed system is designed as a simple menu-driven C++ application. It allows an authorized user to add student details, display the student list, mark students as present or absent for a class session, view individual attendance records and calculate attendance percentages. A basic file-handling approach can be used to retain records between program sessions. The system is intended as a small academic prototype and does not claim to replace a secure, institution-wide attendance platform.

The report discusses the need for the system, existing approaches, requirements, design alternatives, selected methodology, implementation logic and sample validation cases. The expected outcome is a user-friendly prototype that reduces repetitive manual calculations and provides a structured way to maintain attendance data.

Keywords: Student attendance, C++, file handling, attendance percentage, academic record management.

GRAPHICAL ABSTRACT

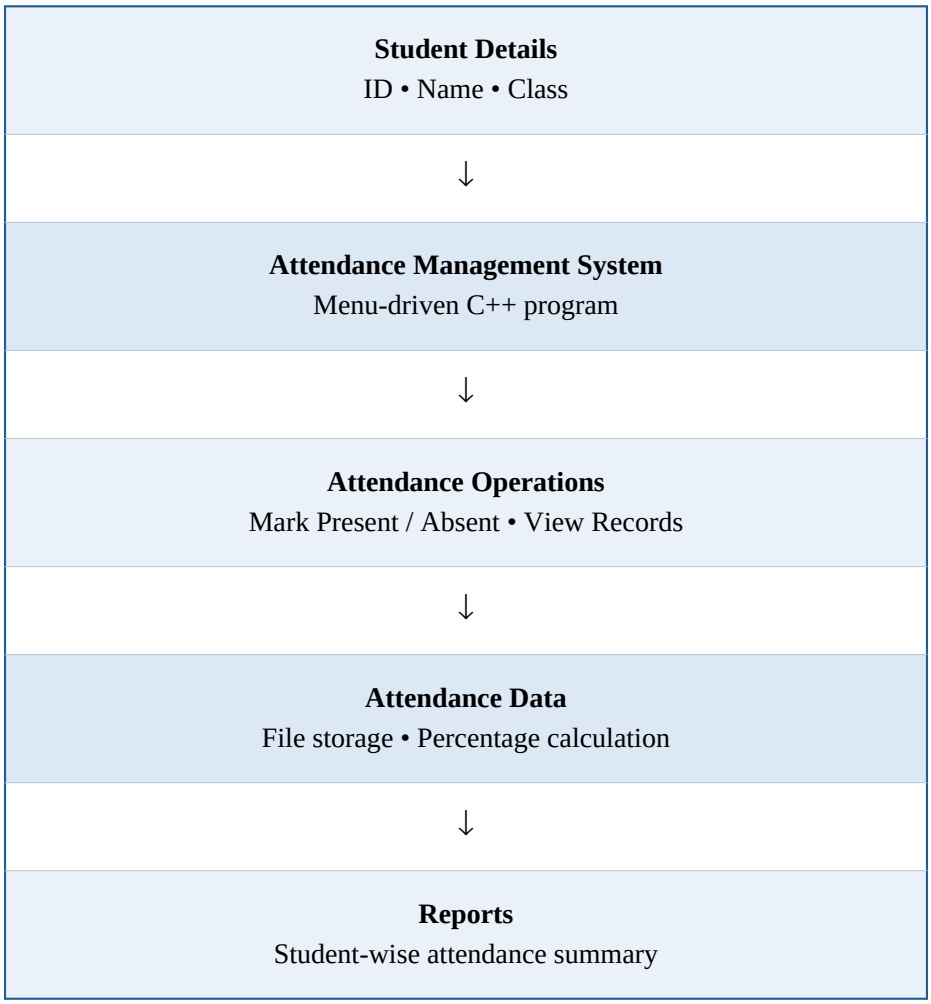


Figure: High-level workflow of the proposed Student Attendance Management System.

ABBREVIATIONS AND SYMBOLS

Abbreviation	Meaning
C++	General-purpose programming language
ID	Unique student identification number
UI	User Interface
CSV	Comma-Separated Values
CRUD	Create, Read, Update and Delete
I/O	Input / Output

Symbols: P = number of sessions present; T = total sessions conducted; Attendance (%) = (P ÷ T) × 100.

CHAPTER 1. INTRODUCTION

1.1 Identification of Client / Need / Relevant Contemporary Issue

Educational institutions need reliable attendance records to monitor student participation, support academic follow-up and prepare administrative summaries. Attendance is commonly recorded through paper registers, spreadsheets or digital platforms. Paper-based recording can be slow in larger classes, while separate spreadsheets may lead to inconsistent formats or duplicated entries if they are not maintained carefully. A small, structured system can help demonstrate how routine attendance tasks may be organized through software.

The intended users are teachers, faculty members or an academic coordinator who need to maintain attendance for a class. The project is an educational prototype; institution-specific policies, privacy requirements and integration with official university databases are outside its present scope.

1.2 Identification of Problem

The central problem is the difficulty of maintaining, retrieving and summarizing student attendance records consistently when information is recorded manually or across disconnected files. Repeated calculations and record searches may consume staff time, and inaccurate or incomplete entries can affect the reliability of attendance summaries.

1.3 Identification of Tasks

The project tasks are: (i) identify users and attendance-record requirements; (ii) define student and session data fields; (iii) compare possible system designs; (iv) design the menu, data structures and processing logic; (v) implement the core operations in C++; (vi) validate the program using sample test cases; and (vii) prepare documentation and a user guide.

1.4 Timeline

Activity	Week 1	Week 2	Week 3	Week 4
Requirement study	✓			
Literature/background study	✓	✓		
System design		✓		
Coding and file handling		✓	✓	
Testing and correction			✓	
Report preparation			✓	✓

Table 1.1: Indicative four-week project timeline. The schedule can be adjusted to match the actual project duration.

1.5 Organization of the Report

Chapter 1 introduces the need, problem, tasks and timeline. Chapter 2 reviews common attendance-recording approaches and defines the project objectives. Chapter 3 explains requirements, constraints, design alternatives and the selected workflow. Chapter 4 describes the implementation approach and validation cases. Chapter 5 presents the conclusion and possible future improvements. References and an appendix/user manual are provided at the end.

CHAPTER 2. LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

Attendance recording has traditionally been performed using paper registers maintained by teachers. As personal computers became common in educational administration, many institutions began using spreadsheets and database applications to store student information and calculate attendance. Web-based and mobile systems later enabled centralized access, automated reporting and, in some cases, integration with student portals. The underlying requirement—accurate recording of presence for each class session—has remained the same, while the tools used to manage the records have evolved.

2.2 Existing Solutions

Paper registers: Simple and inexpensive, but searching, calculating percentages and preparing summaries can be time-consuming.
Spreadsheet-based records: Support sorting, formulas and basic reports, but depend on careful file management and may suffer from accidental editing or inconsistent copies.
Institutional attendance portals: Can provide centralized access, role-based permissions and integration with academic records, but may require infrastructure, training and institutional deployment.
Biometric or card-based systems: Can automate identity capture, but require hardware, maintenance and clear privacy safeguards.

2.3 Comparative Analysis

The following comparison is qualitative and is intended to guide the prototype design; it is not the result of a formal institutional survey.

Method	Cost / setup	Record retrieval	Main limitation
Paper register	Low	Manual search	Slow summaries; physical records
Spreadsheet	Low	Good with organized files	File duplication and editing errors
Attendance portal	Higher setup	Centralized	Needs deployment and support
Proposed C++ prototype	Low for a small class	Menu-based search/display	Limited scalability and security

Table 2.1: Qualitative comparison of attendance management approaches.

2.4 Review Summary

The review indicates that a useful attendance tool should make student identification clear, support quick marking of attendance, retain records and calculate attendance summaries. A small C++ application is suitable for demonstrating these core functions in an academic programming project. It is not intended to provide the full security, scale or integration of a production university portal.

2.5 Problem Definition

The project aims to design and implement a menu-driven application that stores basic student details, records present/absent status for class sessions and displays attendance summaries. The application will use a unique student ID to distinguish records and may use file handling for persistence. It will not include biometric verification, online synchronization, automated messaging or integration with the university database.

2.6 Goals / Objectives

1. Create a student record using a unique ID, name and class/section.
2. Display the list of registered students.
3. Record each student as present or absent for a session.
4. Retrieve attendance records for a selected student.
5. Calculate attendance percentage using present sessions and total recorded sessions.
6. Store and retrieve data through local files in the prototype.
7. Validate normal and incorrect input conditions using test cases.

CHAPTER 3. DESIGN FLOW / PROCESS

3.1 Evaluation and Selection of Specifications / Features

The essential features selected for the prototype are student registration, student-list display, attendance marking, student-wise attendance viewing and percentage calculation. A menu-driven interface is selected because it is straightforward for a beginner-level C++ project and can be operated through keyboard input.

ID	Requirement	Priority
FR-01	Add a student with a unique ID, name and class/section.	High
FR-02	Display all registered students.	High
FR-03	Mark attendance as Present or Absent for a session.	High
FR-04	View a student attendance summary.	High
FR-05	Calculate attendance percentage.	High
FR-06	Save and load records using local files.	Medium
NFR-01	Provide clear menu prompts and input validation.	High
NFR-02	Keep the prototype simple to operate and maintain.	Medium

Table 3.1: Functional and non-functional requirements.

3.2 Design Constraints

The system is designed with the following constraints:

• **Technical:** A C++ compiler and a basic desktop environment are required. The prototype uses local storage rather than a remote database.

• **Economic:** The design should use freely available development tools and avoid paid infrastructure.

• **Usability:** Menus and prompts should be understandable to a first-time user.

• **Privacy:** Student information should be limited to fields needed for attendance. Real deployment would require access control, secure storage and institutional approval.

• **Reliability:** Duplicate student IDs and invalid attendance choices should be checked before saving.

3.3 Analysis of Features and Finalization Subject to Constraints

Features requiring network access, biometric hardware or integration with official academic systems are excluded because they exceed the scope and resources of a small classroom prototype. The final design therefore focuses on core attendance operations, local file storage and simple validation. A login mechanism may be added in a later version; the current prototype assumes operation by an authorized staff member.

3.4 Design Flow: Alternative Designs

Alternative A — Spreadsheet-based system: Student details and attendance are stored in spreadsheet rows and columns. It is quick to prepare and supports formulas, but the workflow depends on manual file handling and may create multiple inconsistent copies.

Alternative B — C++ menu-driven application: Student and attendance records are managed through program functions and stored in local files. It requires programming effort but provides a consistent sequence of operations and demonstrates data structures and file handling.

3.5 Design Selection

Alternative B is selected because the project is intended to demonstrate programming concepts in C++. It provides a dedicated workflow for adding students, marking attendance and generating summaries. The

spreadsheet alternative is easier to start with, but it does not demonstrate the same level of application logic or controlled menu flow.

3.6 Implementation Plan / Methodology

The application follows a simple sequence: start the program, load saved records, display the menu, accept a user choice, perform the selected operation, save changes and return to the menu until the user exits.

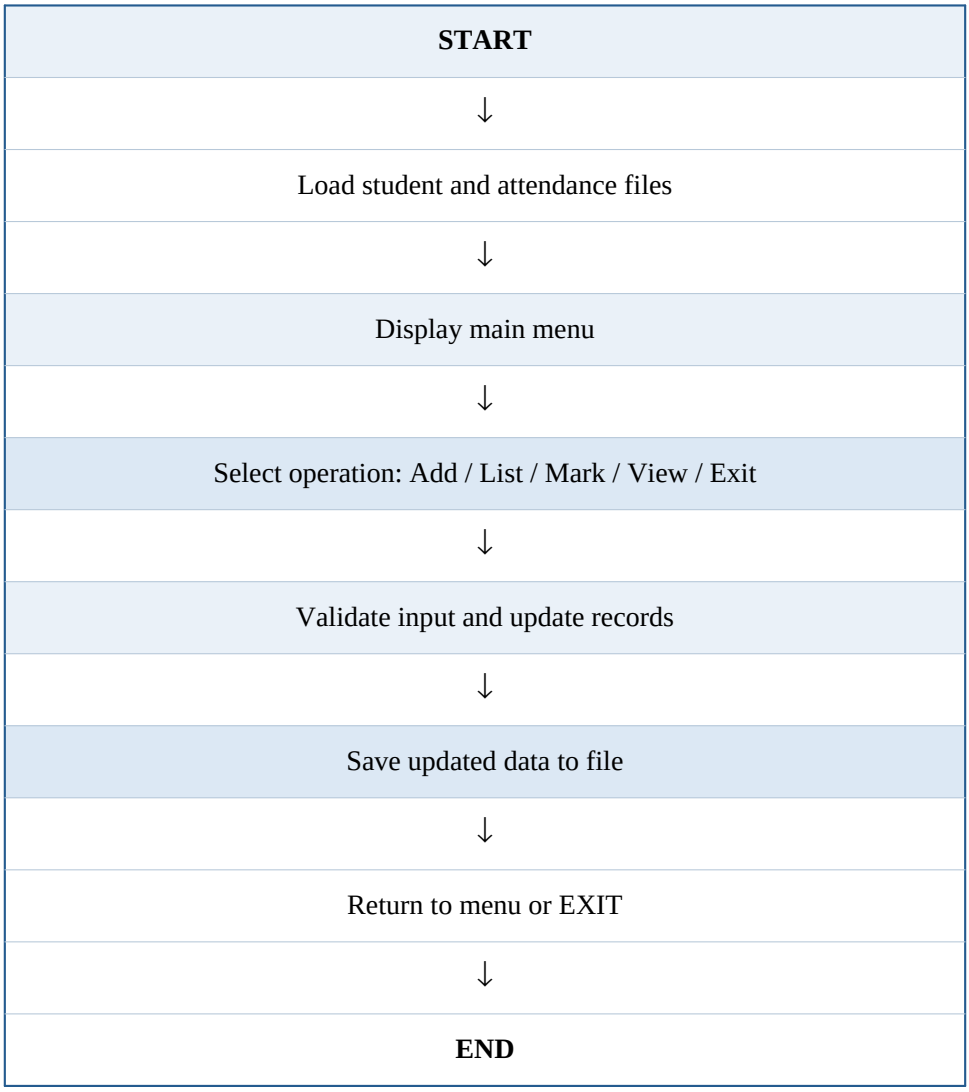


Figure 3.1: Simplified implementation flow of the application.

Attendance calculation: Let P be the number of sessions marked Present and T be the total number of sessions for which attendance has been recorded. If T is greater than zero, attendance percentage is calculated as (P/T) × 100. If T = 0, the system should display “No attendance recorded” rather than attempting division by zero.

CHAPTER 4. RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of Solution

The proposed implementation is a console-based C++ program organized into separate operations. A Student record stores the student ID, name and class/section. Attendance information stores the number of sessions attended and the total number of sessions recorded. A production-ready version should store a dated entry for every class; for a compact prototype, the same totals can be used to demonstrate percentage calculation.

Suggested program modules:

1. **addStudent()** — accepts student details and checks that the ID is not already registered.
2. **displayStudents()** — prints all registered students.
3. **markAttendance()** — records present/absent status for a session and updates the relevant totals.
4. **viewAttendance()** — finds a student by ID and displays the attendance summary.
5. **saveData()** and **loadData()** — write records to and read records from local files.

A simple menu may contain the following options:

STUDENT ATTENDANCE MANAGEMENT SYSTEM
1. Add Student
2. Display Students
3. Mark Attendance
4. View Attendance Summary
5. Exit
Enter your choice: _

Figure 4.1: Illustrative main menu for the console application.

The application should provide clear confirmation messages after adding a student or saving attendance. If a student ID is not found, the program should display an error message and return to the menu. If the user enters an invalid menu choice or attendance status, the input should be rejected and requested again.

4.2 Test Cases and Validation

The following test cases define expected behavior for a sample implementation. They are test scenarios, not a claim that a particular executable was run during preparation of this report.

Test ID	Input / action	Expected result
TC-01	Add a new student ID and valid details.	Student record is added and confirmation is shown.
TC-02	Add an already registered student ID.	Duplicate ID is rejected.
TC-03	Display student list when records exist.	All registered students are displayed.
TC-04	Mark a registered student Present.	Present count and total session count increase.
TC-05	Mark a registered student Absent.	Total session count increases; present count remains unchanged.

TC-06	Search using an unknown student ID.	A “Student not found” message is displayed.
TC-07	View attendance where total sessions = 0.	System displays “No attendance recorded”.
TC-08	View attendance with 8 present out of 10.	Attendance percentage is displayed as 80%.

Table 4.1: Proposed functional test cases and expected results.

Validation should confirm that student IDs remain unique, totals are updated correctly, percentage calculations are accurate and data is retained after restarting the program. The sample calculation 8 ÷ 10 × 100 = 80% can be used to verify the percentage function.

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

The Student Attendance Management System provides a structured approach to maintaining student attendance through a menu-driven C++ application. The proposed design covers the basic activities of student registration, displaying records, marking present or absent status, viewing student-wise summaries and calculating attendance percentages. Local file handling can be used to preserve records between program sessions.

The project demonstrates how a routine academic task can be represented using programming concepts such as structures or classes, functions, conditional statements, loops and file input/output. The expected benefit is a reduction in repetitive manual calculations and easier retrieval of basic attendance information. The system remains a prototype and would need further testing, security controls and institutional review before real-world deployment.

5.2 Future Work

The system can be improved in several ways:

- Add a secure login system with separate teacher and administrator roles.
- Store dated attendance entries in a relational database instead of only local text files.
- Provide class-wise, subject-wise and date-wise reports.
- Add search, edit and correction options with an audit history.
- Develop a graphical user interface or web/mobile interface.
- Introduce backup and restore facilities.
- Apply privacy and security controls, including access permissions and protected storage.
- Integrate with an institution-approved student information system after authorization and testing.

REFERENCES

1. Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley, 2013.

2. cppreference.com, “C++ language and standard library reference,” https://en.cppreference.com/ (accessed October 2026).

3. ISO/IEC 14882, *Programming Languages — C++*, International Organization for Standardization.

4. ISO/IEC 25010, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE)*, International Organization for Standardization, 2011.

5. Pressman, R. S. and Maxim, B. R., *Software Engineering: A Practitioner’s Approach*, 8th Edition, McGraw-Hill, 2014.

Note: The report is a project design and documentation draft. Institution-specific survey statistics, screenshots of a running program, actual test execution evidence and supervisor details should be added only when available.

APPENDIX

A. Sample C++ Logic (Illustrative)

The following compact example demonstrates the attendance percentage calculation. It is not a complete database-backed application; it can be incorporated into the view-summary function of the full program.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    int present, total;

    cout << "Enter present sessions: ";
    cin >> present;
    cout << "Enter total sessions: ";
    cin >> total;

    if (total <= 0 || present < 0 || present > total) {
        cout << "Invalid attendance values.";
    } else {
        double percentage = (present * 100.0) / total;
        cout << fixed << setprecision(2);
        cout << "Attendance: " << percentage << "%";
    }
    return 0;
}
```

B. Design Checklist

- Student ID is unique.
- Required student fields are entered.
- Attendance status accepts only valid choices.
- Present count never exceeds total sessions.
- Zero-session calculation is handled safely.
- Data is saved and can be loaded again.
- Error messages are clear and understandable.

USER MANUAL

The following steps describe how a user would operate the proposed console application after it has been compiled and run.

1. **Start the program:** Open the compiled application. The main menu is displayed.
2. **Add a student:** Select “Add Student” and enter the student ID, name and class/section. The program checks whether the ID already exists.
3. **Display students:** Select “Display Students” to view the registered student list.
4. **Mark attendance:** Select “Mark Attendance”, enter the student ID and choose Present or Absent for the current class session.
5. **View summary:** Select “View Attendance Summary” and enter the student ID. The program displays the number of present sessions, total sessions and attendance percentage.
6. **Exit:** Select “Exit”. The program saves any updated data before closing.

Troubleshooting

• If the student ID is not found, check the ID and register the student before marking attendance.
• If an invalid option is entered, return to the menu and select a valid number.
• If records are not available after restarting, verify that the data files are stored in the expected application folder and that the program has permission to write to that folder.
• Keep a backup of attendance files. Do not share student data with unauthorized persons.

Important note

This report follows the major sections and sequence of the Chandigarh University project-report format supplied for reference. The cover details use the information provided by the requester. Before final submission, confirm the degree/branch wording with the department and add the actual supervisor name, department, signatures, project screenshots, plagiarism report and any institution-required checklist.